

Formal Verification: ML-DSA (FIPS 204) Digital Signatures

Zach Kelling, Lux Industries

December 2025

Abstract

We formalize ML-DSA, the NIST post-quantum digital signature standard based on module lattices. ML-DSA-65 (NIST Level 3, 192-bit security) is the default for Lux validator identities. We state correctness and EUF-CMA security as axioms grounded in the Module-LWE assumption.

1 Introduction

ML-DSA is the lattice-based component of the Lux hybrid signature scheme. It provides post-quantum security based on the hardness of Module-LWE, a well-studied lattice problem. The Fiat-Shamir with Aborts paradigm ensures tight security reductions.

2 Definitions

Definition 1 (SecurityLevel). *Three levels: ML-DSA-44 (128-bit), ML-DSA-65 (192-bit), ML-DSA-87 (256-bit).*

3 Theorems and Axioms

Axiom 2 (mldsa_correctness). *For any valid key pair, sign-then-verify always succeeds.*

Axiom 3 (mldsa_euf_cma). *EUF-CMA security under Module-LWE: no PPT/quantum adversary can forge a signature on a new message.*

Theorem 4 (sig_size_bounded). *All ML-DSA signature sizes are ≤ 5000 bytes.*

Theorem 5 (default_level_3). *ML-DSA-65 signature size is exactly 3309 bytes.*

Theorem 6 (all_sigs_bounded). *All signature sizes fit in a network packet (< 5000 bytes).*

4 Lean Source

/-

ML-DSA (FIPS 204) Signature Correctness

Models the ML-DSA precompile (src/precompiles/mldsa/).

ML-DSA is the NIST post-quantum digital signature standard,

based on the Fiat-Shamir with Aborts paradigm over module lattices.

Maps to:

- *src/precompiles/mldsa/: Go implementation*
- *luxfi/crypto: ML-DSA package*

Properties:

- *Correctness: sign then verify always succeeds*
- *EUF-CMA: existential unforgeability under chosen message attack*
- *Signature size: 3309 bytes for ML-DSA-65 (security level 3)*

-/

```
import Mathlib.Data.Nat.Defs
```

```
import Mathlib.Tactic
```

```
namespace Crypto.MLDSA
```

```
/-- ML-DSA security levels -/
```

```
inductive SecurityLevel where
```

```
  | level2 : SecurityLevel -- ML-DSA-44 (128-bit)
```

```
  | level3 : SecurityLevel -- ML-DSA-65 (192-bit)
```

```
  | level5 : SecurityLevel -- ML-DSA-87 (256-bit)
```

```
/-- Signature sizes by security level -/
```

```
def sigSize (l : SecurityLevel) : Nat :=
```

```
  match l with
```

```
  | .level2 => 2420
```

```
  | .level3 => 3309
```

```
  | .level5 => 4627
```

```
/-- Public key sizes -/
```

```
def pkSize (l : SecurityLevel) : Nat :=
```

```
  match l with
```

```
  | .level2 => 1312
```

```
  | .level3 => 1952
```

```
  | .level5 => 2592
```

```
/-- Abstract sign and verify functions for ML-DSA -/
```

```
axiom mldsa_sign : SecurityLevel → Nat → Nat → Nat
```

```
axiom mldsa_verify : SecurityLevel → Nat → Nat → Nat → Bool
```

```
axiom mldsa_correctness :
```

```
  (l : SecurityLevel) (sk pk msg : Nat),
```

```
    -- For a valid key pair (sk, pk), sign then verify always succeeds
```

```
    mldsa_verify l pk msg (mldsa_sign l sk msg) = true
```

```
/-- EUF-CMA security under Module-LWE assumption -/
```

```
axiom mldsa_euf_cma :
```

```
  (l : SecurityLevel) (pk msg : Nat) (forgery : Nat),
```

```
    -- No PPT/quantum adversary can forge a signature on a new message
```

```
    -- even after seeing signatures on chosen messages (Module-LWE hardness)
```

```
    mldsa_verify l pk msg forgery = true →
```

```
    (sk : Nat), forgery = mldsa_sign l sk msg
```

```

/-- Signature size is bounded -/
theorem sig_size_bounded (l : SecurityLevel) :
  sigSize l 5000 := by
  cases l <|> simp [sigSize]

/-- ML-DSA-65 is the default for Lux (NIST Level 3, 192-bit security) -/
theorem default_level_3 : sigSize .level3 = 3309 := rfl

/-- Public key size at level 3 -/
theorem pk_size_level3 : pkSize .level3 = 1952 := rfl

/-- Security levels are ordered -/
theorem level2_lt_level3 : sigSize .level2 < sigSize .level3 := by simp [
  sigSize]
theorem level3_lt_level5 : sigSize .level3 < sigSize .level5 := by simp [
  sigSize]

/-- All signature sizes fit in a network packet (< 5KB) -/
theorem all_sigs_bounded : l : SecurityLevel, sigSize l < 5000 := by
  intro l; cases l <|> simp [sigSize]

end Crypto.MLDSA

```

5 References

Source code: <https://github.com/luxfi/proofs/blob/main/lean/Crypto/MLDSA.lean>

White papers: <https://papers.lux.network>

Related white papers:

- [lux-ethfalcon-post-quantum.tex](#)
- [lux-evm-precompiles.tex](#)